

Merkblatt Meld

Textdateien vergleichen – auch über Git: Arbeitsstand, Staging, Commits, Branches.

Welche Fassung will ich mit welcher vergleichen – und welcher Befehl öffnet genau das in Meld?

Meld zeigt zwei oder drei Fassungen einer Datei nebeneinander und markiert die Unterschiede. Zusammen mit Git kannst du damit jede frühere Fassung einer Datei gegen jede andere stellen.

1 Einrichtung (einmalig)

Windows Installieren mit Scoop

```
scoop bucket add extras
scoop install extras/meld
Get-Command meld   # Kontrolle
```

Scoop legt einen Starter `meld` im `PATH` an.

Linux Installieren als Flatpak

```
flatpak install flathub org.gnome.meld
```

Ein Flatpak legt keinen Befehl `meld` an, und Git legt seine Vergleichskopien in `/tmp` ab, das ein Flatpak nicht in jedem Fall sieht. Deshalb zwei Zusatzschritte:

```
# Linux 1: Zugriff auf /tmp erlauben (sonst zeigt Meld über Git leere Seiten)
flatpak override --user --filesystem=/tmp org.gnome.meld

# Linux 2: Starter "meld" anlegen, damit Shell und Git das Flatpak finden
mkdir -p ~/.local/bin
cat > ~/.local/bin/meld <<'EOF'
#!/bin/sh
exec flatpak run org.gnome.meld "$@"
EOF
chmod +x ~/.local/bin/meld
meld --version     # Kontrolle; ~/.local/bin muss im PATH stehen
```

Git mit Meld verbinden (Windows und Linux gleich)

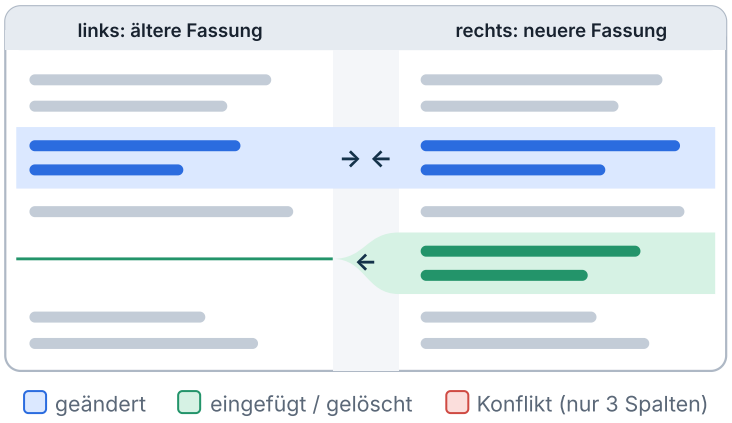
```
git config --global diff.tool meld
git config --global difftool.prompt false       # nicht vor jeder Datei nachfragen
git config --global merge.tool meld
git config --global mergetool.meld.useAutoMerge true
git config --global mergetool.keepBackup false  # keine *.orig-Dateien zurücklassen
```

Git ruft einfach `meld` auf und findet damit den Scoop-Starter bzw. den Starter aus `~/.local/bin`. Kontrolle: `git difftool --tool-help` muss `meld` unter den verfügbaren Werkzeugen nennen.

2 Meld ohne Git

ZIEL	BEFEHL
zwei Dateien vergleichen	`meld alt.md neu.md`
drei Dateien vergleichen	`meld a.md b.md c.md`
zwei Ordner vergleichen	`meld ordner1 ordner2`
mehrere Vergleiche als Tabs	`meld --diff a.md b.md --diff c.md d.md`
Vergleich in laufendem Meld öffnen	`meld --newtab alt.md neu.md`

3 Bedienung in Meld

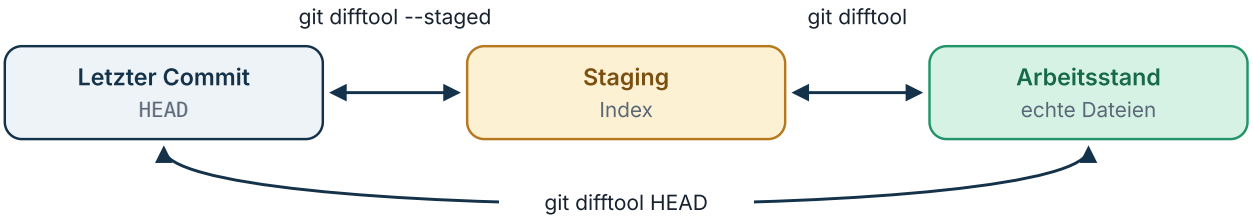


TASTE	WIRKUNG
`Alt` + `↓` / `↑`	nächste / vorherige Änderung
`Alt` + `→` / `←`	Änderung nach rechts / links übernehmen
`Strg` + `S`	aktuelle Datei speichern
`Strg` + `Z`	rückgängig
`Strg` + `F`	suchen
`Strg` + `R`	Vergleich neu laden
`Strg` + `W`	Vergleich schließen

- Die Pfeile zwischen den Spalten übernehmen einen Block in die andere Datei. Mit gedrückter `Umschalt`-Taste wird daraus Löschen, mit `Strg` Einfügen statt Ersetzen.
- Beide Seiten sind editierbar. Du kannst direkt im Vergleich tippen.

4 Meld mit Git: welche Fassung gegen welche?

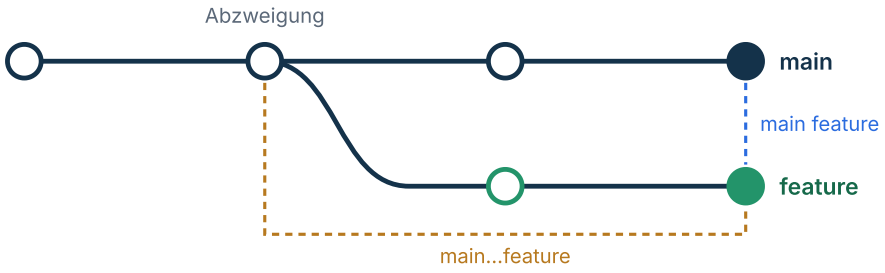
Grundform: `git difftool [<alt>] [<neu>] -- <datei>`. Ohne `-- <datei>` öffnet Git nacheinander jede geänderte Datei; Meld schließen ruft die nächste auf.



Die drei Stände in Git und der Befehl, der je zwei davon vergleicht

VERGLEICH	BEFEHL
Eigene, noch nicht committete Änderungen	
Arbeitsstand gegen Staging (nicht gestagte Änderungen)	`git difftool -- datei.md`
Staging gegen letzten Commit	`git difftool --staged -- datei.md`
Arbeitsstand gegen letzten Commit	`git difftool HEAD -- datei.md`
Commits	
Arbeitsstand gegen einen älteren Commit	`git difftool a1b2c3d -- datei.md`
Arbeitsstand gegen den Stand vor 3 Commits	`git difftool HEAD~3 -- datei.md`
zwei Commits gegeneinander	`git difftool a1b2c3d e4f5a6b -- datei.md`
was ein bestimmter Commit geändert hat	`git difftool a1b2c3d~1 a1b2c3d -- datei.md`

Branches, Remote, Stash	
Arbeitsstand gegen anderen Branch	`git difftool feature -- datei.md`
zwei Branches gegeneinander	`git difftool main feature -- datei.md`
was `feature` seit der Abzweigung von `main` geändert hat	`git difftool main...feature -- datei.md`
lokaler Stand gegen Remote (nach `git fetch`)	`git difftool HEAD origin/main -- datei.md`
letzter Commit gegen Stash	`git difftool HEAD 'stash@{0}' -- datei.md`
zwei verschiedene Dateien aus verschiedenen Ständen	`git difftool main:alt.md feature:neu.md`



`main feature` vergleicht die beiden Spitzen · `main...feature` vergleicht die Abzweigung mit der Spitze von `feature`

Links steht immer die ältere bzw. zuerst genannte Fassung, rechts die neuere.

Speichern möglich

Ist eine Seite dein **Arbeitsstand**, liegt dort die echte Datei. Änderungen in Meld landen nach `Strg` + `S` direkt im Arbeitsverzeichnis.

Geht verloren

Fassungen aus Commits, Branches oder dem Staging sind **temporäre Kopien**. Was du dort änderst, geht beim Schließen verloren.

5 Verlauf einer Datei nachverfolgen

Meld vergleicht Fassungen. Welche Commits überhaupt interessant sind, findest du vorher mit Git:

FRAGE	BEFEHL
Welche Commits haben die Datei geändert?	`git log --oneline --follow -- datei.md`
… mit den Änderungen als Text	`git log -p --follow -- datei.md`
Wer hat welche Zeile zuletzt geändert?	`git blame datei.md`
… nur Zeilen 10 bis 20	`git blame -L 10,20 datei.md`
Wie hat sich Zeile 10 bis 20 entwickelt?	`git log -L 10,20:datei.md`
Wann kam ein bestimmter Text hinzu oder verschwand?	`git log --oneline -S"Suchtext" -- datei.md`
Wie sah die Datei in einem Commit aus?	`git show a1b2c3d:datei.md`

Typischer Ablauf

```
git log --oneline --follow -- datei.md      # Commit heraussuchen, z. B. a1b2c3d
git difftool a1b2c3d~1 a1b2c3d -- datei.md  # dessen Änderung in Meld ansehen
git difftool a1b2c3d -- datei.md            # oder: damals gegen heute
```

`--follow` verfolgt die Datei auch über Umbenennungen hinweg.

6 Ganze Stände vergleichen

```
git difftool --dir-diff                # alle nicht gestagten Änderungen
git difftool --dir-diff main feature   # zwei Branches komplett
git difftool --dir-diff HEAD~5         # Arbeitsstand gegen Stand vor 5 Commits
```

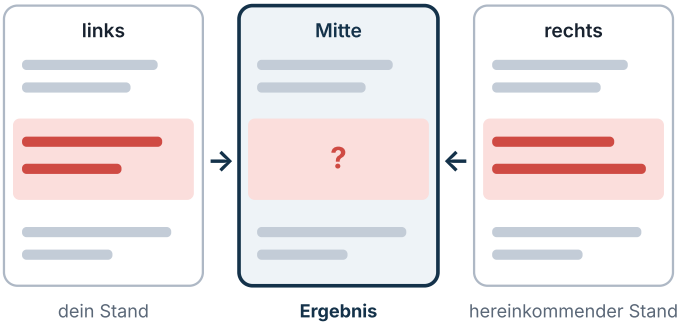
`--dir-diff` (kurz `-d`) öffnet Meld einmal mit einer Ordneransicht aller geänderten Dateien. Ein Doppelklick öffnet den Vergleich einer Datei. Das ist bei vielen Dateien angenehmer als ein Meld-Fenster nach dem anderen.

Alternative ohne `git difftool`: im Repo `meld .` aufrufen. Meld zeigt dann selbst alle geänderten Dateien gegenüber dem letzten Commit.

7 Merge-Konflikte lösen

```
git merge feature
# CONFLICT (content): Merge conflict in datei.md
git mergetool
```

1. Mit `Alt`+`↓` von Konflikt zu Konflikt springen.
2. Den passenden Block von links oder rechts in die Mitte übernehmen oder die Mitte von Hand bearbeiten.
3. Mitte mit `Strg`+`S` speichern, Meld schließen.
4. `git status` prüfen, dann `git commit`.



Mit `mergetool.meld.useAutoMerge true` hat Meld alles, was sich ohne Konflikt zusammenführen ließ, schon in die Mitte übernommen. Übrig bleiben nur die echten Konflikte.

8 Stolperfallen

- **Umbenannte Dateien:** `git difftool HEAD~5 -- neu.md` findet die alte Fassung nicht, wenn die Datei damals anders hieß. Beide Namen angeben: `git difftool HEAD~5 -- alt.md neu.md`.
- **Leere linke oder rechte Seite:** Die Datei existiert in diesem Stand nicht (neu angelegt, gelöscht oder anderer Name).
- **Flatpak: Meld zeigt über Git leere Seiten:** Der Zugriff auf `/tmp` fehlt, siehe Einrichtung.
- **Meld öffnet sich nicht:** Es gibt keinen Unterschied. `git difftool` startet das Werkzeug nur für Dateien, die `git diff` als geändert ansieht.
- **PowerShell und `>`:** `git show a1b2c3d:datei.md > alt.md` schreibt in Windows PowerShell 5.1 eine UTF-16-Datei. Für Vergleiche besser direkt `git difftool` verwenden statt alte Fassungen zu exportieren.
- **`stash@{0}` in PowerShell:** immer in einfache Anführungszeichen setzen.
- **Nur Zeilenenden unterschiedlich:** siehe Arbeitsblatt „Git-Zeilenenden" in der Wissensbasis.

Merksatz: Erst mit `git log` den Stand finden, dann mit `git difftool <alt> <neu> -- <datei>` in Meld ansehen. Speichern lässt sich nur die Seite, die dein Arbeitsstand ist.